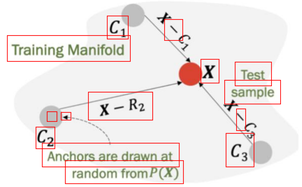
Accurate And Scalable Estimation Of Epistemic Uncertainty For Graph Neural Networks

Puja Trivedi[1], Mark Heimann[2], Rushil Anirudh[2], Danai Koutra[1] Jayaraman J. Thiagarajan[2]

Stochastic Centering: Single Model Ensembles

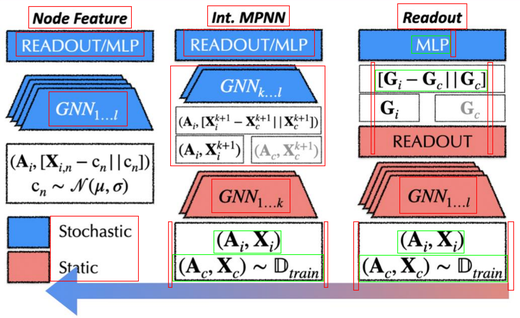
NTKs for (Graph) Neural Networks are not shift invariant → The variability over shifts captures epistemic uncertainty!

- Stochastic Centering creates a ***relative representation*** for an input, ***x***, in terms of a random ***anchor***, ***c***.
- During training, the anchor is ***randomized***, thus implicitly sampling ***different hypotheses.***



- During testing, predictions are computed over ***multiple*** anchors.
- ***Discrepancy*** across anchors choices captures **epistemic uncertainty**!

G Δ-UQ: Stochastic Centering for GNNs



Anchored Inference

Prediction:

$$\mu(y|\mathcal{G}_i) = \frac{1}{K}\sum_{k=1}^{K} f_\theta([\mathcal{G}_i, \mathbf{c}_k])$$

Uncertainty:

$$\sigma(y|\mathcal{G}_i) = \sqrt{\frac{1}{K-1}\sum_{k=1}^{K}(f_\theta([\mathcal{G}_i, \mathbf{c}_k]) - \mu)^2}$$

Calibrated Prediction:

$$\mu_{calib} = \mu(1 - \sigma)$$

- Since graphs are variable sized and discrete, we introduce three anchoring strategies tailored for GNNs.
- Induce **varying levels of stochasticity**, trading off hypothesis diversity and the semantic diversity of the anchoring distribution.

We propose G Δ-UQ, a novel training paradigm for improving calibration with GNNs

- ✓ Scalable and Lightweight
- ✓ Robust to Distribution Shifts
- ✓ Supports Pretrained Models
- ✓ Easy to Implement

```
class GNN_ReadOutAnc:
    def __init__(GAN):
        self.MPNN1 = MPNN(InputDim,HDim1)
        self.MPNN2 = MPNN(HDim1, HDim2)
        self.MPNN3 = MPNN(HDim2, HDim3)
        self.READOUT = READOUTLayer()
        #Corres. to GDUQLayer!
        self.classifier = Linear(HDim3 * 2, NumClass)

    def forward(X,Adj):
        h = self.MPNN1(X,Adj)
        h = self.MPNN2(h,Adj)
        h = self.MPNN3(h,Adj)
        g = self.READOUT(h,Batch) #OUT: (B, HDim3)
        C = SHUFFLE(g, dim=0) #(B, HDim3)
        gAnc = CONCAT([g-C,C],dim=1)#(B, HDim3 x 2)
        pred = self.classifier(gAnc) #Out: (B, NumClass)
        return pred
```

G Δ-UQ is easy to implement, with only a few changes to standard GNNs !

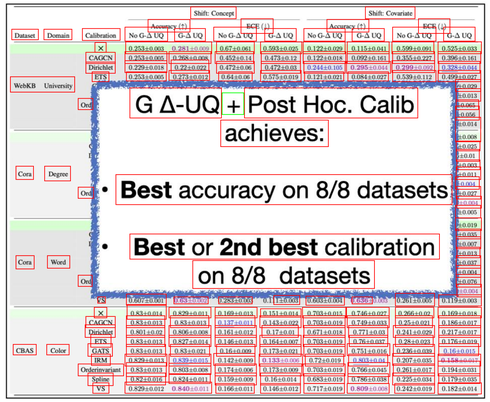
```
#training
for (X,Y) in trainloader:
    C = create_anchors(n = K)
    Batch_AnchAnc = CONCAT([X-C,C], dim = 1)
    preds = model(X, Anc)
    loss = criterion(preds,Y)

#inference
anc_preds = []
testAncs = create_anchors(n = K)
for A in testAncs:
    G_anc = CONCAT([I-A,A], dim = 1)
    preds = model(G_anc)
    anc_preds.append(preds)
P = CONCAT(anc_preds, dim = 0)
mu = MEAN(P, dim = 0)
var = STDDEV(P, dim = 0)
```

Improved Node Classification Calibration

We evaluate G Δ-UQ with node feature anchoring on:

- 4 Datasets
- 2 Distribution shifts
- 8 *Post Hoc* Calibration

G Δ-UQ matches or surpasses accuracy and calibration of the vanilla GNN on 8/8 datasets.

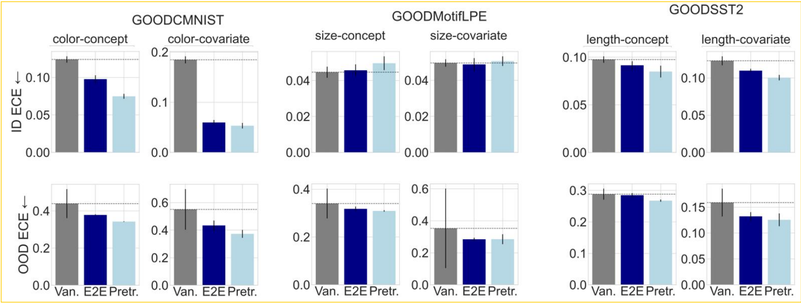


G Δ-UQ + Post Hoc. Calib achieves:

- **Best** accuracy on 8/8 datasets
- **Best** or **2nd best** calibration on 8/8 datasets

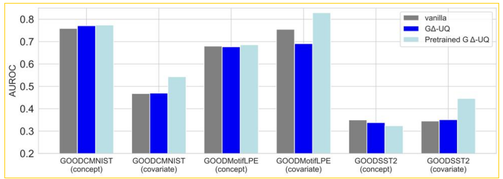
Better Graph Classification Calibration

G Δ-UQ Improves Calibration of Pretrained GNNs

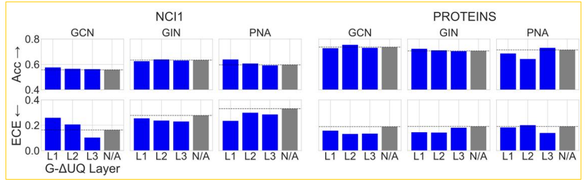


G Δ-UQ Estimates are Effective in Safety Critical Tasks

- G Δ-UQ achieves the best OOD detection AUROC on 5/6 datasets.
- G Δ-UQ also performs well on the generalization gap prediction tasks.



G Δ-UQ Improves Graph Classification Calibration with Different MPNNs



- G Δ-UQ is flexible to message passing layer.
- Required level of stochasticity may vary for dataset/GNN.

Check out the G Δ-UQ Family!

Try E Δ-UQ for link prediction calibration!